

二次予選 解説

2019年12月12日・情報オリンピック日本委員会

第19回日本情報オリンピック 二次予選

問題 1 ポスター (Poster)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

まず、JOI 君は以下の順番で作業を行うとしてよい。

- ポスターを回転させる。
- その後、各マス目を塗り替える。

よって、最初にポスターを回転させる量を固定すれば、各マス目を塗り替える作業だけを行う場合を考えればよくなる。

その場合にかかる時間は、元のポスターと新しいポスターで色が異なるマス目の個数と等しいので、これは盤面のサイズに対して線形時間で求めることができる。

以上より、この問題は $O(N^2)$ の計算量で解くことができる。

原文：<https://www2.ioi-jp.org/joi/2019/2020-yo2/2020-yo2-t1-review.html>

問題 2 いちご (Strawberry)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

いちご i が赤くなる時刻 T_i は $\max(A_i, T_i)$ に置き換えても問題ない。なぜなら、地点 A_i に到達するには少なくとも A_i 秒かかるため、時刻 A_i より早く赤くなるケースは時刻 A_i に赤くなるとしても答えには影響ないからである。以降この置き換えを行った上で議論をすすめる。

いちご i を赤い状態で収穫して地点 0 に戻るのは最速でも時刻 $A_i + T_i$ である。よって $1 \leq i \leq N$ に対する $(A_i + T_i)$ の最大値を M とすると、この問題に対する答えは M 以上にならなければならない。

一番東にあるいちごをいちご e とする。まず地点 A_e まで行き、そのまま地点 0 へ直行するが、もしまだ青いいちごが実をつけている地点 (地点 A_e も含む) に到着したら、それが赤くなるまで待つ、という動き方を考える。このとき、地点 0 に到着するのはちょうど時刻 M となる。よって答えは M になる。

原文：<https://www2.ioi-jp.org/joi/2019/2020-yo2/2020-yo2-t2-review.html>

問題 3 桁和 (Digit Sum)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

JOI 君が初めに持っていた可能性のある整数を N -到達数 と呼ぶことにする。 1 以上 N 以下の整数 x が N -到達数 である必要十分条件は下記の通りである：

- $x = N$ または $x + d(x)$ が N -到達数 となる。

ただし、 $d(x)$ は x を十進法で表したときの各桁の和である。

したがって、 $x = N, N - 1, \dots, 1$ の順に N -到達数 であるかを判定すればよい。 $d(x)$ は $O(\log_{10} N)$ で求めることができるので、全体の計算量は $O(N \log_{10} N)$ となる。

原文：<https://www2.ioi-jp.org/joi/2019/2020-yo2/2020-yo2-t3-review.html>

問題 4 テンキー (Tenkey)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

小課題 1

小課題 1 では, JOI 君が入力すべき整数は $100000 \times n + R$ (n は 0 以上の整数) の形で表すことができる.

このような整数の下 5 桁は R で固定されているため, 操作回数を最小化するためには R を入力するのが最適であることがわかる.

R を入力するのに必要な操作の回数を計算するには, R の各桁について, 「その数字が印字されているキーまで移動し, キーを押す」ために必要な操作の数をそれぞれ計算し, 足し合わせればよい.

小課題 2

答えは最大で 30 になり, 操作の種類は $2 \sim 5$ 通りあるので, 単純な全探索では時間内に答えを出すことができない.

そこで, 状態をまとめることを考える. 具体的には, (カーソルの位置, 入力した数字を M で割った余り) の組を状態とする. このとき求めるべきものは, カーソルの位置が 0 , 余りが 0 である状態から, 余りが R である状態にするために必要な最小の操作回数である.

これは, それぞれの状態を頂点とし, 可能な操作に対応した辺を張ったグラフ上で, 幅優先探索を用いて最短経路を計算することで求めることができる. 状態数は $O(M)$, それぞれの状態で可能な操作は $O(1)$ 通りであるので, 上記のグラフの辺数は $O(M)$ となる. よって幅優先探索は $O(M)$ で実行でき, これは全てのケースを正解するのに十分高速である.

原文: <https://www2.ioi-jp.org/joi/2019/2020-yo2/2020-yo2-t4-review.html>

問題 5 じゃんけん式 (Rock-Scissors-Paper Expression)

配点 100点

時間制限 2 sec

メモリ制限 1024 MB

解説

小課題 1

小課題 1 では、`?` の個数が 8 個以下であることがわかる。単純な解法として、`?` に `R`，`S`，`P` のどれを割り当てるかすべての場合を考え (最大 $3^8 = 6561$ 通り)，それぞれについて式を計算するという方法が考えられる。割り当てをすべて試すのは，再帰関数を用いたり，0 以上 $3^{(? \text{ の個数})}$ 未満の整数を対応させたりといった実装方法がある。問題となるのは式の計算の方法，すなわち構文解析である。様々な構文解析の方法のうち，ここでは 2 つを紹介する。

動的計画法

文字列中の区間についての動的計画法を用いて構文解析を行うことができる。これは与えられた BNF をそのままの形で用いる方法である。

文字列 `E` の `i` 文字目から `j` 文字目までの部分文字列について，以下の 4 つの値を保持する：

- `<factor>` であるか否か
- `<term>` であるか否か
- `<expression>` であるか否か
- 計算結果 (`<expression>` である場合)

例えば，部分文字列 `R+S` に対してはこれらは順に `false`，`true`，`true`，`R` となる。これらの値は，区間の長さが短い順に埋めていくことで計算できる。例えば，`i` 文字目から `j` 文字目までが `<term>` であるのは，それが `<factor>` であるか，ある `k` ($i < k < j$) が存在して「`i` 文字目から `k-1` 文字目までが `<term>` である」「`k` 文字目が `*`」「`k+1` 文字目から `j` 文字目までが `<term>`」を満たす場合である。

計算すべき値が $O(N^2)$ 個あり，それぞれの計算が $O(N)$ 時間でできるので，構文解析にかかる時間計算量は $O(N^3)$ となる。

再帰下降法

本問の文法を言い換えると，

- `<expression>` は `<term>` に `+` `<term>` または `-` `<term>` を 0 個以上続けたもの
- `<term>` は `<factor>` に `*` `<factor>` を 0 個以上続けたもの

であることがわかる。このことを用いると，文字列のある位置から `<factor>` を読む処理ができれば，文字列のある位置からできるだけ長い `<term>` を読む処理ができ，それを用いて文字列のある位置からできるだけ長い `<expression>` を読む処理ができる。`<factor>` を読む処理は `(` を見た場合にそこから `<expression>` を読むことになるので，3 つの関数を用いた相互再帰で書ける。疑似コードは以下のようになる：

```

function expression():
  value := term()
  while 次の文字が '+' または '-':
    1 文字読み進める
    value := value + term() または value - term()
  return value

function term():
  value = factor()
  while 次の文字が '*':
    1 文字読み進める
    value := value * factor()
  return value

function factor():
  if 次の文字が 'R' または 'S' または 'P':
    1 文字読み進める
    return 'R' または 'S' または 'P'
  else:
    1 文字読み進める (開き括弧)
    value := expression()
    1 文字読み進める (閉じ括弧)
    return value

```

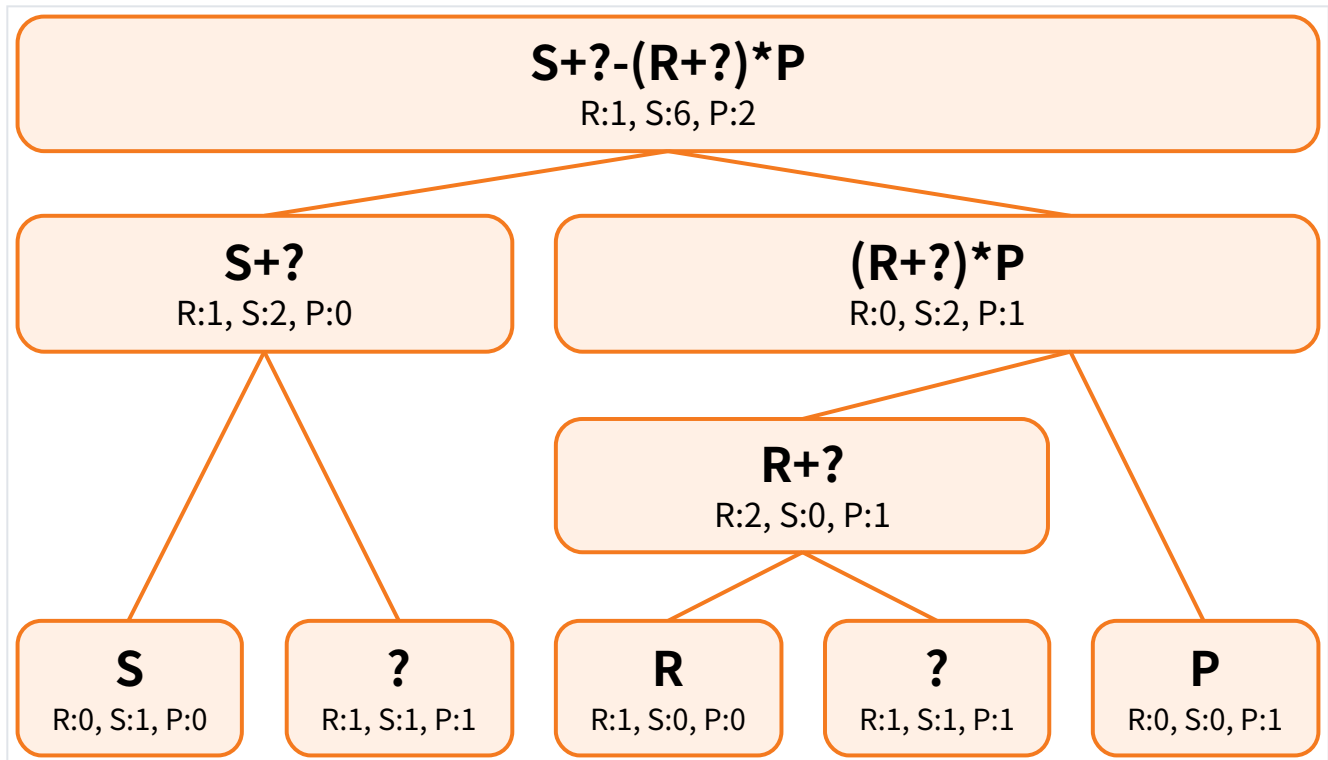
文字列の「今見ている場所」を管理し (具体的には、グローバル変数や戻り値などを用いて実装する), 関数 `expression`, `term`, `factor` はそれぞれ今見ている場所からできるだけ長い `<expression>`, `<term>`, `<factor>` を読み, 見ている場所を進め, 計算結果を返す関数になっている. 最初に文字列の先頭を見ている状態で関数 `expression` を呼ぶことで, 式の計算結果がわかる (構文にエラーがなければ見ている場所は文字列の末尾となる).

このようなアルゴリズムは再帰下降法と呼ばれる. 文字列の各位置を読む回数が `expression`, `term`, `factor` のそれぞれにおいて高々 1 回ずつであるから, 時間計算量は $O(N)$ である. 再帰の深さも (よって空間計算量も) $O(N)$ である.

小課題 2, 3

⑦ への割り当てをすべて試さずに答えを求めるためには, 動的計画法の考え方が必要である. 文字列 `E` の部分文字列として表される式のそれぞれについて, 計算結果が `R`, `S`, `P` になるような割り当て方が何通りあるかをそれぞれ求める. そのような式同士に `+`, `-`, `*` の演算を行ったものについても計算結果ごとに割り当て方が何通りあるか求めることができる.

この考え方は以下の図のように表すことができる (入力例 1 に対応している). このような木は構文木と呼ばれる.



入力例 1 に対応する構文木

例えば、式 $S+?$ の計算結果を R , S , P にする方法はそれぞれ 1 通り, 2 通り, 0 通りであり、式 $(R+?)*P$ の計算結果を R , S , P にする方法はそれぞれ 0 通り, 2 通り, 1 通りである。これらに演算 $-$ を行くと、 $S-S=S$ となるのが $2 \times 2 = 4$ 通り, $S-P=P$ となるのが $2 \times 1 = 2$ 通りというように、式 $S+?-(R+?)*P$ の計算結果を R , S , P にする方法が 9 回の掛け算でわかる。

この方法を、動的計画法による構文解析にそのままあてはめる (計算結果の代わりに何通りかを求めていく) と、 $O(N^3)$ 時間でこの問題が解ける (小課題 2)。

R , S , P を値とする計算の代わりに、「割り当て方が何通りかを表す整数の 3 つ組」を値に持つ計算を考えると、再帰下降法などによって $O(N)$ 時間でこの問題が解ける (小課題 3)。

原文 : <https://www2.ioi-jp.org/joi/2019/2020-yo2/2020-yo2-t5-review.html>

出典：情報オリンピック日本委員会「第19回日本情報オリンピック JOI 2019/2020 二次予選競技課題」。原資料は CC BY-SA 4.0 で提供されています。

本PDFは各解説ページを問題順に再構成したものです。

<https://www2.ioi-jp.org/joi/2019/2020-yo2/index.html>